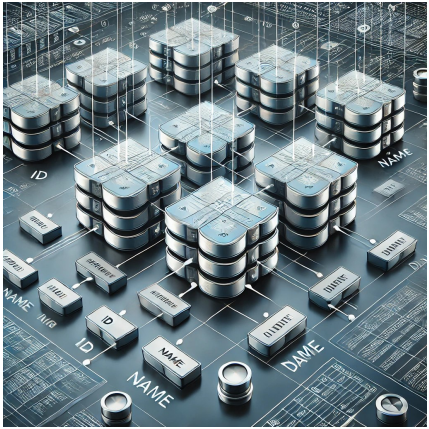
LMU LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

Datenbanksysteme

Prof. Dr. Marcus Paradies

Version: 16/10/2026 08:22



AI-generated visual representation of a database system

Kapitel 1

Grundlagen von Datenbanksystemen

Motivation

Daten als das neue Öl des 21. Jahrhunderts

- Dienste basieren auf Daten statt auf physischen Ressourcen (z.B. besitzt Uber keine Taxis, AirBnB keine Häuser oder Zimmer)
- Geschäftlicher Wert entsteht vollständig aus Daten (sei es durch Generierung, Sammlung oder Bereitstellung) ⇒ Daten müssen um jeden Preis geschützt und gut verwaltet werden
- Unternehmen können ohne sichere, zugängliche und flexible Daten nicht überleben
- Datenbanksysteme bieten eine universelle, standardisierte Möglichkeit, Daten sicher zu speichern



Quelle: https://www.economist.com/leaders/2017/05/06/the-worlds-most-valuable-resource-is-no-longer-oil-but-data

Beispiel: Amazon Prime Day



Beispiel: Amazon Prime Day



Amazon DynamoDB – DynamoDB powers multiple high-traffic Amazon properties and systems including Alexa, the Amazon.com sites, and all Amazon fulfillment centers. Over the course of Prime Day, these sources made trillions of calls to the DynamoDB API. DynamoDB maintained high availability while delivering single-digit millisecond responses and peaking at 126 million requests per second.

Amazon Aurora – On Prime Day, 5,835 database instances running the PostgreSQL-compatible and MySQL-compatible editions of Amazon Aurora processed 318 billion transactions, stored 2,140 terabytes of data, and transferred 836 terabytes of data.

Quelle: https://aws.amazon.com/de/blogs/aws/prime-day-2023-powered-by-aws-all-the-numbers/

Quelle: https://fitsmallbusiness.com/amazon-prime-day-statistics/

Wir wollen lernen, **wie** man ein Datenbanksystem auf **effektive** Weise nutzt

Wir wollen verstehen, wie Datenbanksysteme **funktionieren**, um ihre Leistung optimieren zu können

Aber Moment mal..., warum Datenbanksysteme?

Welche Probleme treten bei alternativen Lösungen auf?

- Angenommen, wir wollen Informationen über Studierende, Kurse und Einschreibungen verwalten

Aber Moment mal..., warum Datenbanksysteme?

Welche Probleme treten bei alternativen Lösungen auf?

Angenommen, wir wollen Informationen über Studierende, Kurse und Einschreibungen verwalten

Wir starten mit einem „Straw-Man Proposal“: Python (how hard can it be...)

Aber Moment mal..., warum Datenbanksysteme?

Welche Probleme treten bei alternativen Lösungen auf?

Angenommen, wir wollen Informationen über Studierende, Kurse und Einschreibungen verwalten

Wir starten mit einem „Straw-Man Proposal“: Python (how hard can it be...)

Anforderungen:

Wir wollen Daten speichern, sie einlesen, Abfragen darüber formulieren und Daten ändern.

Umfrage: Welche Probleme könnten auftreten, wenn Sie anstatt eines Datenbanksystems ein selbstgeschriebenes Programm zur Datenverwaltung nutzen?

pingo.coactum.de/042471



Aber Moment mal..., warum Datenbanksysteme?

Welche Probleme treten bei alternativen Lösungen auf?

Aber Moment mal..., warum Datenbanksysteme?

Welche Probleme treten bei alternativen Lösungen auf?

$\Rightarrow$ **Datenredundanzen und -inkonsistenzen**

Aber Moment mal..., warum Datenbanksysteme?

Welche Probleme treten bei alternativen Lösungen auf?

⇒ **Datenredundanzen und -inkonsistenzen**

⇒ **Eingeschränkte Zugriffsmöglichkeiten auf Daten**

Aber Moment mal..., warum Datenbanksysteme?

Welche Probleme treten bei alternativen Lösungen auf?

$\Rightarrow$ **Datenredundanzen und -inkonsistenzen**

$\Rightarrow$ **Eingeschränkte Zugriffsmöglichkeiten auf Daten**

$\Rightarrow$ **Probleme beim Mehrbenutzerzugriff**

Aber Moment mal..., warum Datenbanksysteme?

Welche Probleme treten bei alternativen Lösungen auf?

⇒ **Datenredundanzen und -inkonsistenzen**

⇒ **Eingeschränkte Zugriffsmöglichkeiten auf Daten**

⇒ **Probleme beim Mehrbenutzerzugriff**

⇒ **Datenverluste**

Aber Moment mal..., warum Datenbanksysteme?

Welche Probleme treten bei alternativen Lösungen auf?

⇒ **Datenredundanzen und -inkonsistenzen**

⇒ **Eingeschränkte Zugriffsmöglichkeiten auf Daten**

⇒ **Probleme beim Mehrbenutzerzugriff**

⇒ **Datenverluste**

⇒ **Verletzung der Datenintegrität**

Aber Moment mal..., warum Datenbanksysteme?

Welche Probleme treten bei alternativen Lösungen auf?

⇒ **Datenredundanzen und -inkonsistenzen**

⇒ **Eingeschränkte Zugriffsmöglichkeiten auf Daten**

⇒ **Probleme beim Mehrbenutzerzugriff**

⇒ **Datenverluste**

⇒ **Verletzung der Datenintegrität**

⇒ **Sicherheitsprobleme**

Aber Moment mal..., warum Datenbanksysteme?

Welche Probleme treten bei alternativen Lösungen auf?

⇒ **Datenredundanzen und -inkonsistenzen**

⇒ **Eingeschränkte Zugriffsmöglichkeiten auf Daten**

⇒ **Probleme beim Mehrbenutzerzugriff**

⇒ **Datenverluste**

⇒ **Verletzung der Datenintegrität**

⇒ **Sicherheitsprobleme**

⇒ **Hoher Entwicklungsaufwand**

Was ist ein Datenbanksystem?

- System zur Speicherung und Verwaltung von Daten
- **Datenbanksystem** (DBS): **Datenbank** (DB) & **Datenbankmanagementsystem** (DBMS)

Was ist ein Datenbanksystem?

- System zur Speicherung und Verwaltung von Daten
- **Datenbanksystem** (DBS): **Datenbank** (DB) & **Datenbankmanagementsystem** (DBMS)

Datenbank

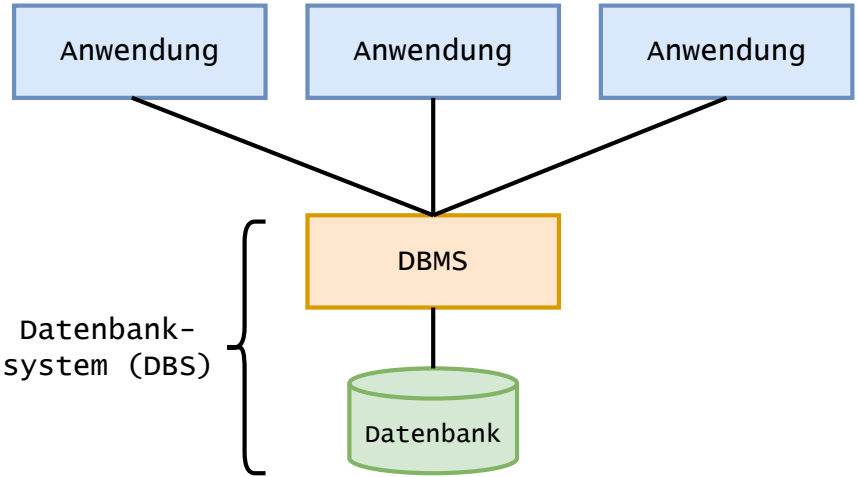
- Eine (typischerweise sehr große) integrierte Sammlung miteinander verknüpfter Daten, die persistent gespeichert sind
- Daten beschreiben Informationen und Aktivitäten über eine oder mehrere miteinander verbundene Objekte/Entitäten (Ausschnitt aus der realen Welt)
- Beispiel: Eine Universitätsdatenbank enthält Informationen über
  - Entitäten (z.B. Studierende, Lehrveranstaltungen, Fakultäten, ...)
  - Beziehungen zwischen Entitäten (z.B. "Bill belegt die Veranstaltung Datenbanksysteme")

Was ist ein Datenbanksystem?

Datenbankmanagementsystem

- Eine Sammlung von Softwarepaketen, die zur Speicherung, zum Zugriff und zur Verwaltung von Datenbanken entwickelt wurden

Was ist ein Datenbanksystem?



Was ist der Zweck eines Datenbanksystems?

⇒ Codds zwölf Regeln

Was ist der Zweck eines Datenbanksystems?

⇒ Codds zwölf Regeln

Datenintegration: Alle Daten werden einheitlich verwaltet

Was ist der Zweck eines Datenbanksystems?

⇒ Codds zwölf Regeln

Datenintegration: Alle Daten werden einheitlich verwaltet

Effizienter Datenzugriff: Datenbanksprachen werden bereitgestellt, um Daten zu speichern, darauf zuzugreifen und sie zu verwalten

Was ist der Zweck eines Datenbanksystems?

⇒ Codds zwölf Regeln

Datenintegration: Alle Daten werden einheitlich verwaltet

Effizienter Datenzugriff: Datenbanksprachen werden bereitgestellt, um Daten zu speichern, darauf zuzugreifen und sie zu verwalten

Datenkatalog: Enthält alle Informationen über Objekte und Strukturen der Datenbank (Metadaten)

Was ist der Zweck eines Datenbanksystems?

⇒ Codds zwölf Regeln

Datenintegration: Alle Daten werden einheitlich verwaltet

Effizienter Datenzugriff: Datenbanksprachen werden bereitgestellt, um Daten zu speichern, darauf zuzugreifen und sie zu verwalten

Datenkatalog: Enthält alle Informationen über Objekte und Strukturen der Datenbank (Metadaten)

Benutzer-/Anwendungssichten: Unterschiedliche Sichten für verschiedene Benutzer und Anwendungen

Was ist der Zweck eines Datenbanksystems?

⇒ Codds zwölf Regeln

Datenintegration: Alle Daten werden einheitlich verwaltet

Effizienter Datenzugriff: Datenbanksprachen werden bereitgestellt, um Daten zu speichern, darauf zuzugreifen und sie zu verwalten

Datenkatalog: Enthält alle Informationen über Objekte und Strukturen der Datenbank (Metadaten)

Benutzer-/Anwendungssichten: Unterschiedliche Sichten für verschiedene Benutzer und Anwendungen

Integritätsbedingungen: werden vom DBMS sichergestellt

Was ist der Zweck eines Datenbanksystems?

⇒ Codds zwölf Regeln

Datenintegration: Alle Daten werden einheitlich verwaltet

Effizienter Datenzugriff: Datenbanksprachen werden bereitgestellt, um Daten zu speichern, darauf zuzugreifen und sie zu verwalten

Datenkatalog: Enthält alle Informationen über Objekte und Strukturen der Datenbank (Metadaten)

Benutzer-/Anwendungssichten: Unterschiedliche Sichten für verschiedene Benutzer und Anwendungen

Integritätsbedingungen: werden vom DBMS sichergestellt

Sicherheitsmechanismen: zum Schutz der Daten vor Sicherheitsbedrohungen

Was ist der Zweck eines Datenbanksystems?

⇒ Codds zwölf Regeln

Datenintegration: Alle Daten werden einheitlich verwaltet

Effizienter Datenzugriff: Datenbanksprachen werden bereitgestellt, um Daten zu speichern, darauf zuzugreifen und sie zu verwalten

Datenkatalog: Enthält alle Informationen über Objekte und Strukturen der Datenbank (Metadaten)

Benutzer-/Anwendungssichten: Unterschiedliche Sichten für verschiedene Benutzer und Anwendungen

Integritätsbedingungen: werden vom DBMS sichergestellt

Sicherheitsmechanismen: zum Schutz der Daten vor Sicherheitsbedrohungen

Transaktionen: fassen mehrere Operationen auf Daten zu logischen Einheiten zusammen

Was ist der Zweck eines Datenbanksystems?

⇒ Codds zwölf Regeln

Datenintegration: Alle Daten werden einheitlich verwaltet

Effizienter Datenzugriff: Datenbanksprachen werden bereitgestellt, um Daten zu speichern, darauf zuzugreifen und sie zu verwalten

Datenkatalog: Enthält alle Informationen über Objekte und Strukturen der Datenbank (Metadaten)

Benutzer-/Anwendungssichten: Unterschiedliche Sichten für verschiedene Benutzer und Anwendungen

Integritätsbedingungen: werden vom DBMS sichergestellt

Sicherheitsmechanismen: zum Schutz der Daten vor Sicherheitsbedrohungen

Transaktionen: fassen mehrere Operationen auf Daten zu logischen Einheiten zusammen

Synchronisation: gleichzeitiger Benutzertransaktionen

Was ist der Zweck eines Datenbanksystems?

⇒ Codds zwölf Regeln

Datenintegration: Alle Daten werden einheitlich verwaltet

Effizienter Datenzugriff: Datenbanksprachen werden bereitgestellt, um Daten zu speichern, darauf zuzugreifen und sie zu verwalten

Datenkatalog: Enthält alle Informationen über Objekte und Strukturen der Datenbank (Metadaten)

Benutzer-/Anwendungssichten: Unterschiedliche Sichten für verschiedene Benutzer und Anwendungen

Integritätsbedingungen: werden vom DBMS sichergestellt

Sicherheitsmechanismen: zum Schutz der Daten vor Sicherheitsbedrohungen

Transaktionen: fassen mehrere Operationen auf Daten zu logischen Einheiten zusammen

Synchronisation: gleichzeitiger Benutzertransaktionen

Wiederherstellung: von Daten nach Systemabstürzen

Was ist der Zweck eines Datenbanksystems?

⇒ Codds zwölf Regeln

Datenintegration: Alle Daten werden einheitlich verwaltet

Effizienter Datenzugriff: Datenbanksprachen werden bereitgestellt, um Daten zu speichern, darauf zuzugreifen und sie zu verwalten

Datenkatalog: Enthält alle Informationen über Objekte und Strukturen der Datenbank (Metadaten)

Benutzer-/Anwendungssichten: Unterschiedliche Sichten für verschiedene Benutzer und Anwendungen

Integritätsbedingungen: werden vom DBMS sichergestellt

Sicherheitsmechanismen: zum Schutz der Daten vor Sicherheitsbedrohungen

Transaktionen: fassen mehrere Operationen auf Daten zu logischen Einheiten zusammen

Synchronisation: gleichzeitiger Benutzertransaktionen

Wiederherstellung: von Daten nach Systemabstürzen

Ad-hoc: Anfragen, Berichtsgenerierung, Schnittstellen zu anderen Datenbanksystemen, Schnittstellen zur Anwendungsprogrammierung, ...

Geschichte des Datenmanagements

Die 60er-Jahre

- Anfang der 60er: Einfache Dateien, anwendungsspezifische Datenorganisation (geräteabhängig, redundant, inkonsistent)
- Ende der 60er: Dateiverwaltungssysteme (SAM, ISAM) mit Dienstprogrammen (z.B. Sortierung) (geräteunabhängig, aber weiterhin redundant und inkonsistent)
- Datenbanksysteme basierend auf hierarchischem und Netzwerkmodell
  - Zeigerstrukturen zwischen Datenelementen
  - Schwache Trennung zwischen interner und konzeptioneller Ebene
  - Navigierender Datenzugriff
  - Trennung zwischen Datenbanksprache und Programmiersprache

Geschichte des Datenmanagements

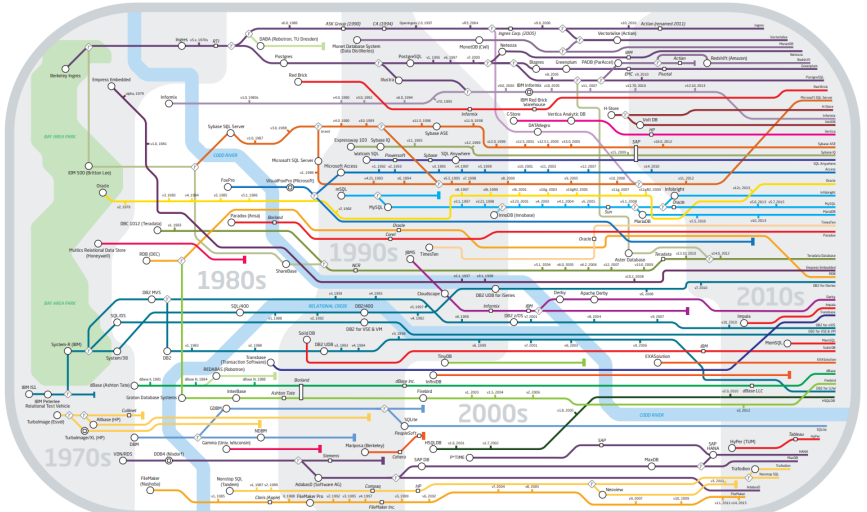
Die 70er- und 80er-Jahre

- 70er: Datenbanksysteme (geräte- und datenunabhängig, keine Redundanzen, konsistent)
- **Relationale Datenbanksysteme**
  - Daten in Tabellen (Relationen)
  - Drei-Ebenen-Architektur (Sicht-, logische und physische Ebene)
  - Deklarative Datenbanksprachen
  - Trennung von Datenbanksprache und Programmiersprache

Geschichte relationaler Datenbanksysteme

- **1970** Ted Codd (IBM) → Relationales Modell als konzeptionelle Grundlage relationaler DBS
- **1974** System R (IBM) → erster Prototyp eines RDBMS
  - Nur zwei Module: RDS, RSS; ca. 80.000 LOC (PL/1, PL/S, Assembler), ca. 1,2 MB Codegröße
  - Abfragesprache SEQUEL
  - Erste Installation 1977
- **1975** University of California at Berkeley (UCB) → Ingres
  - Abfragesprache QUEL
  - Vorgänger von Postgres, Sybase, ...
- **1979** Oracle Version 2
- **Heute** Marktvolumen von ca. 24 Milliarden US-Dollar

Stammbaum Relationaler Datenbanksysteme


1970s
1980s
1990s
2000s
2010s
BAY AREA PARK
CODD RIVER
RELATIONAL CREEK
Berkeley Ingres
Ingres
IDS
SWIM (Ridehrstein, TU Dresden)
Planet Database System (Data Dictionaries)
Prolifics
Red Brick
Informix
Sybase SQL Server
IDM 500 (DB+Kon Lee)
Oracle
DBC 1012 (Teradata)
Multics Relational Data Store (Honeywell)
RDB (DEC)
DB2 MVS
SQL/DS
DB2
System R (IBM)
System/38
IBM AS
IBM Pinpoint Relational Tech Vehicle
Gamma (Univ. Wisconsin)
NDBM
DB/M
DDM
VSAM/DDS
FileMaker (Rocketfish)
Nonstop SQL (Tandem)
Class (Apple)
FileMaker Pro
FileMaker Inc.
Empress Embedded
Informix
Sybase SQL Server
Sybase IQ
Sybase ASE
Microsoft SQL Server
Microsoft Access
VisualFoxPro (Microsoft)
FoxPro
Paradox (Ansa)
Borland
dBase
dBase Inc.
Ashton Tate
Interbase
Firebird
Arago
Ingres Corp. (2005)
Microsoft Cairo
Netezza
PostgreSQL
Illustra
Illustra Information Technologies
Illustra (Informix)
Postgres
Postgres95
PostgreSQL
Mariposa (Berkeley)
Cohera
Truviso
Greenplum
EMC (Pivotal)
Pivotal
Redshift (Amazon)
Vectorwise (Actian)
Actian (renamed 2011)
Ingres
VectorH
Paraccel
Amazon Redshift
Vertica
Vertica Analytic DB
HP Vertica
HP Neoview
HP NonStop SQL
HP Trafodion
Trafodion
Hi Store
Visit DB
SAP
SAP HANA
SAP Sybase ASE
SAP Sybase IQ
SQL Anywhere
Sybase
Watcom SQL
Powersoft
Watcom
Adaptive Server Enterprise
Adaptive Server Anywhere
Sybase SQL Anywhere
Sybase 10
Sybase 11
Sybase 12
Sybase 15
Sybase 16
Sybase 17
Sybase 15.7
Sybase 15.5
Sybase 15.0.3
Sybase 15.0.2
Sybase 15.0.1
Sybase 15.0
Sybase 12.5
Sybase 12.0
Sybase 11.9.2
Sybase 11.9
Sybase 11.5
Sybase 11.0
Sybase 10.0
Sybase 4.9.2
Sybase 4.9.1
Sybase 4.2
Sybase 4.0
Sybase 3.0
Sybase 2.0
Sybase 1.0
Sybase SQL Server
Sybase 10.0
Sybase 4.2
Sybase 4.0
Sybase 3.0
Sybase 2.0
Sybase 1.0

Entwicklungstrends: Heute

Neue Hardware-Architekturen

- Mehrkernprozessoren, Hauptspeicher im Terabyte-Bereich $\Rightarrow$ In-Memory-Datenbanksysteme (z.B. SAP HANA)
- Hardwarebeschleunigung durch Co-Prozessoren (z.B. FPGAs oder GPUs)
- Tiefere Speicherhierarchien (z.B. nichtflüchtiger Speicher, SSDs / Flash)

Entwicklungstrends: Heute

Unterstützung spezieller Anwendungen

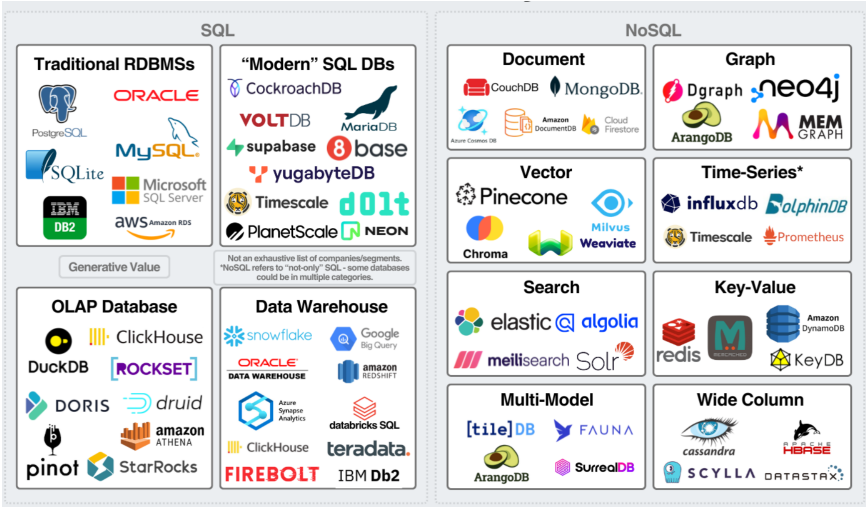
- Cloud-Datenbanksysteme: Hosting von Datenbanksystemen, skalierbare Datenmanagementlösungen (z.B. Amazon RDS, Microsoft Azure)
- Datenstromverarbeitung: Online-Verarbeitung von Live-Daten, z.B. Börsendaten, Sensordaten, RFID-Daten (StreamBase, MS StreamInsight, IBM Infosphere Streams)
- Big Data: Verarbeitung von Daten im Petabyte-Bereich durch hochskalierbare, parallele Datenverarbeitung und -analyse (z.B. Hadoop, Hive, Google Spanner & F1)

Entwicklungstrends: Heute

NoSQL-Datenbanksysteme (not-only SQL")

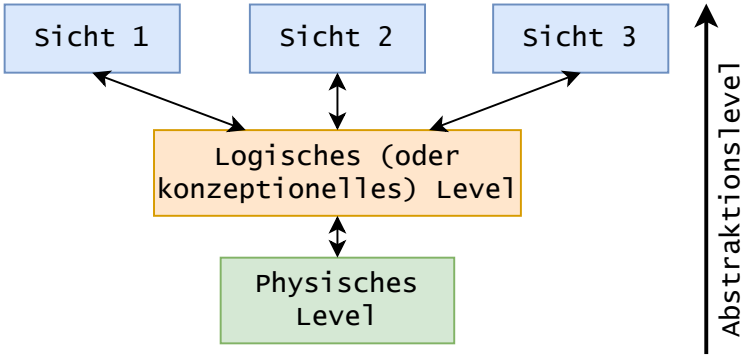
- Nicht-relationale Datenbanksysteme mit flexiblem Schema (dokumentenorientiert)
- „Leichtgewichtig“ durch Weglassen komplexer Funktionen relationaler DBMS wie Transaktionen und deklarativer Anfrageverarbeitung
- Beispiele: CouchDB, MongoDB, Cassandra usw.

Das Datenbank-Ökosystem



Unterschiedliche Sichten auf Daten

Ein wesentliches Ziel eines DBMS ist es, den Benutzern eine abstrakte Sicht auf die Daten zu bieten, d.h. es verbirgt Details darüber, wie Daten auf einem Computer gespeichert und verwaltet werden.



Unterschiedliche Sichten auf Daten (Fortsetzung)

Sichtebene (View Level) beschreibt, wie Benutzer und Anwendungen die Daten sehen

Logische Ebene (Logical Level) (auch konzeptionelle Ebene genannt) beschreibt, welche Daten im DBS gespeichert werden (in Form von Entitäten und Beziehungen); Schwerpunkt liegt auf der logischen Struktur der Datenbank

Physische Ebene (Physical Level) beschreibt, wie Datensätze tatsächlich gespeichert sind und wie Dateien und Indizes organisiert und verwendet werden

Abstraktion wird erreicht, indem jede Ebene mit einem **Datenbankschema** beschrieben wird, das wiederum auf einem **Datenmodell** basiert

Schemata & Instanzen

- Ein **Datenbankschema** ist die Beschreibung einer bestimmten Sammlung von Daten unter Verwendung eines gegebenen **Datenmodells**
- Eine **Instanz** eines **Datenbankschemas** ist der tatsächliche Inhalt der Datenbank zu einem bestimmten Zeitpunkt
- Schemata existieren auf verschiedenen Abstraktionsebenen:
  - **Konzeptionelles (oder logisches) Schema**: bildet typischerweise die Grundlage für das Design einer Datenbank (Schwerpunkt dieses Kurses)
  - **Sichten (oder externe Schemata)**: meist während der Anforderungsanalyse definiert (müssen oft in ein gemeinsames konzeptionelles Schema integriert werden)
  - **Physisches Schema**: Speicherstrukturen, die mit den Relationen verbunden sind

Datenunabhängigkeit

- Fähigkeit, die Definition eines Schemas auf einer Ebene zu ändern, ohne dass dies Auswirkungen auf die Schemadefinitionen auf höheren Ebenen hat
- Wird durch die Nutzung der drei Ebenen der Datenabstraktion erreicht (⇒ **Drei-Ebenen-Schemaarchitektur**)

Logische Datenunabhängigkeit

- Fähigkeit, das logische Schema zu ändern, ohne Anwendungsprogramme neu schreiben zu müssen

Physische Datenunabhängigkeit

- Fähigkeit, das physische Schema zu ändern, ohne das logische Schema oder Anwendungen anpassen zu müssen (mitunter notwendig zur Leistungsverbesserung)

Datenmodelle

Ein Datenmodell ist eine Sammlung von Konzepten zur Beschreibung von:

- Daten und Beziehungen zwischen Daten
- Datensemantik und Datenbeschränkungen (Constraints)

Beispiele (konzeptionelle Modelle):

- Entity-Relationship (ER)-Modell
- Semantisches Datenmodell
- Objektorientiertes (OO) Modell (z.B. mittels UML)

Beispiele (logische Modelle):

- Relationales Modell
- Objektorientiertes oder objektrelationales Modell
- Netzwerkmodell
- Hierarchisches Modell
- XML/JSON

Datenbanksprachen

Data Definition Language (DDL)

- Beschreibungssprache (Notation) zur Definition eines Datenbankschemas; beinhaltet Syntax und Semantik
- DDL-Compiler erzeugt eine Menge von Tabellen, die im Datenkatalog (Data Dictionary) des DBMS gespeichert werden (enthält Metadaten – „Daten über Daten“)
- Datenspeicher- und Definitionssprache – spezielle Form der DDL zur Festlegung von Speicherstrukturen und Zugriffsmethoden, die vom DBS verwendet werden

Beispiel (SQL)

```
CREATE TABLE Assistent (
    PersNr INTEGER,
    Name VARCHAR(80),
    Abteilung VARCHAR(200)
);
```

Datenbanksprachen

Data Manipulation Language (DML)

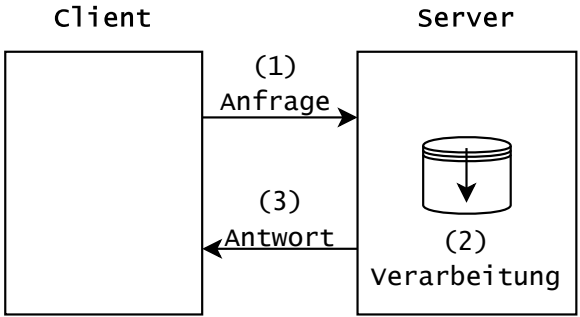
- Sprache zur Abfrage und Manipulation von Daten, die entsprechend einem zugrunde liegenden Datenmodell organisiert sind
- Zwei Klassen von Sprachen:
  - Prozedural – Benutzer gibt an, **wie** die gewünschten Daten abgerufen werden
  - Deklarativ – Benutzer gibt an, **welche** Daten benötigt werden, aber nicht, wie sie abgerufen werden sollen
  - Deklarativ $\equiv$ nicht-prozedural

Beispiel (SQL)

```
INSERT INTO KUNDEN VALUES ('Scott','Tiger','Munich') -- Einfuegeoperation
SELECT * FROM KUNDEN WHERE VORNAME = 'Scott'     -- Abfrage
```

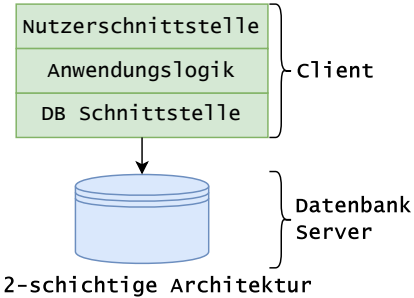
Anwendungsarchitekturen

– Architektur von Datenbankanwendungen basiert typischerweise auf dem Client-Server-Modell (der Server hostet das Datenbanksystem)

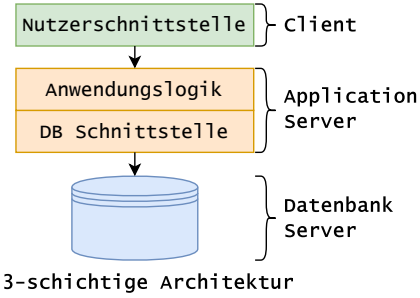


Anwendungsarchitekturen

- Trennung der Funktionalitäten einer Anwendung:
  - Benutzeroberfläche und Interaktion
  - Anwendungslogik (Business Logic)
  - Datenverwaltungsfunktionalität (Speicherung, Verwaltung, Zugriff)



2-schichtige Architektur



3-schichtige Architektur

Zusammenfassung

- Datenbanksysteme sind die Grundlage jeder datenbezogenen Aufgabe, bei der Daten gespeichert, verwaltet und abgerufen werden müssen
- Sie werden in völlig unterschiedlichen Anwendungskontexten eingesetzt (z.B. Finanzen, Automobil, ERP/CRM, Gesundheitswesen, Versicherungen usw.)
- Datenbanksysteme werden in der Cloud, lokal (on-premise), am Edge und in eingebetteten Systemen verwendet
- Datenbanksysteme speichern Daten **zuverlässig** und **sicher** und ermöglichen gleichzeitig einen **leistungsstarken** Datenzugriff

Zusammenfassung

- Datenbanksysteme sind die Grundlage jeder datenbezogenen Aufgabe, bei der Daten gespeichert, verwaltet und abgerufen werden müssen
- Sie werden in völlig unterschiedlichen Anwendungskontexten eingesetzt (z.B. Finanzen, Automobil, ERP/CRM, Gesundheitswesen, Versicherungen usw.)
- Datenbanksysteme werden in der Cloud, lokal (on-premise), am Edge und in eingebetteten Systemen verwendet
- Datenbanksysteme speichern Daten **zuverlässig** und **sicher** und ermöglichen gleichzeitig einen **leistungsstarken** Datenzugriff

In diesem Kurs wollen wir (1) lernen, **wie man Datenbanksysteme effizient nutzt** und (2) verstehen, **wie Datenbanksysteme aufgebaut sind**, um deren Leistung optimal auszuschöpfen.

Kursübersicht

Kapitel	Inhalt
1	Grundlagen von Datenbanksystemen
2	Relationales Modell
3	Relationale Algebra
4	Relationen-Kalkül
5	SQL
6	Konzeptionelles Datenbankdesign
7	Theorie relationaler Datenbanken
8	Speicher- und Dateistrukturen
9	Transaktionsverarbeitung
10	Anfrageverarbeitung
11	Ausblick